

# MapReduce

The slide features a dark green header area. Below the 'MapReduce' title, there is a thick green horizontal bar. Underneath this bar, the background is white. On the right side of the white area, there are several thin, horizontal green lines of varying lengths, creating a stepped or layered effect.

- **MapReduce** es un modelo de programación y una implementación asociada para el procesamiento y generación de grandes conjuntos de datos.
- Los programas escritos en este estilo funcional son paralelizados automáticamente y se ejecutan en grandes clústeres.
- La computación consiste en aplicar una operación ***map*** a cada “record” lógico del input a fin de generar un conjunto intermedio de pares key/value y luego aplicar una operación ***reduce*** a todos los valores que compartan un mismo key.

# Modelo de Programación

- Se toma como input un conjunto de pares key/value y se produce como output un conjunto de pares key/value.
- Se requieren dos funciones definidas por el usuario: Map y Reduce.
- **Función map:** Toma como input un par y genera un conjunto de pares key/value intermedios. La librería MapReduce agrupa todos los valores intermedios asociados al mismo key intermedio y se los pasa a la función reduce.
- **Función reduce:** Acepta una key intermedia y un conjunto de valores para dicha key. Se unen todos los valores para formar un conjunto de valores posiblemente más pequeño. Por lo general se genera sólo un valor como output o ninguno por cada invocación de la función reduce.

# Ejemplo sencillo

- Problema: contar la ocurrencia de cada palabra en un larga colección de documentos.
- Pseudocódigo:

```
map(String key, String value):  
    // key: document name  
    // value: document contents  
    for each word w in value:  
        EmitIntermediate(w, "1");
```

```
reduce(String key, Iterator values):  
    // key: a word  
    // values: a list of counts  
    int result = 0;  
    for each v in values:  
        result += ParseInt(v);  
    Emit(AsString(result));
```

# Acerca de los dominios

- Conceptualmente, las funciones map y reduce poseen ciertos tipos asociados:

$$\begin{array}{lll} \text{map} & (k1, v1) & \rightarrow \text{list}(k2, v2) \\ \text{reduce} & (k2, \text{list}(v2)) & \rightarrow \text{list}(v2) \end{array}$$

- Los keys y values de input se obtienen de un dominio distinto al de los keys y values intermedios.
- Los keys y values intermedios son del mismo dominio que los keys y values que se obtienen como output final.

# Ejemplos adicionales

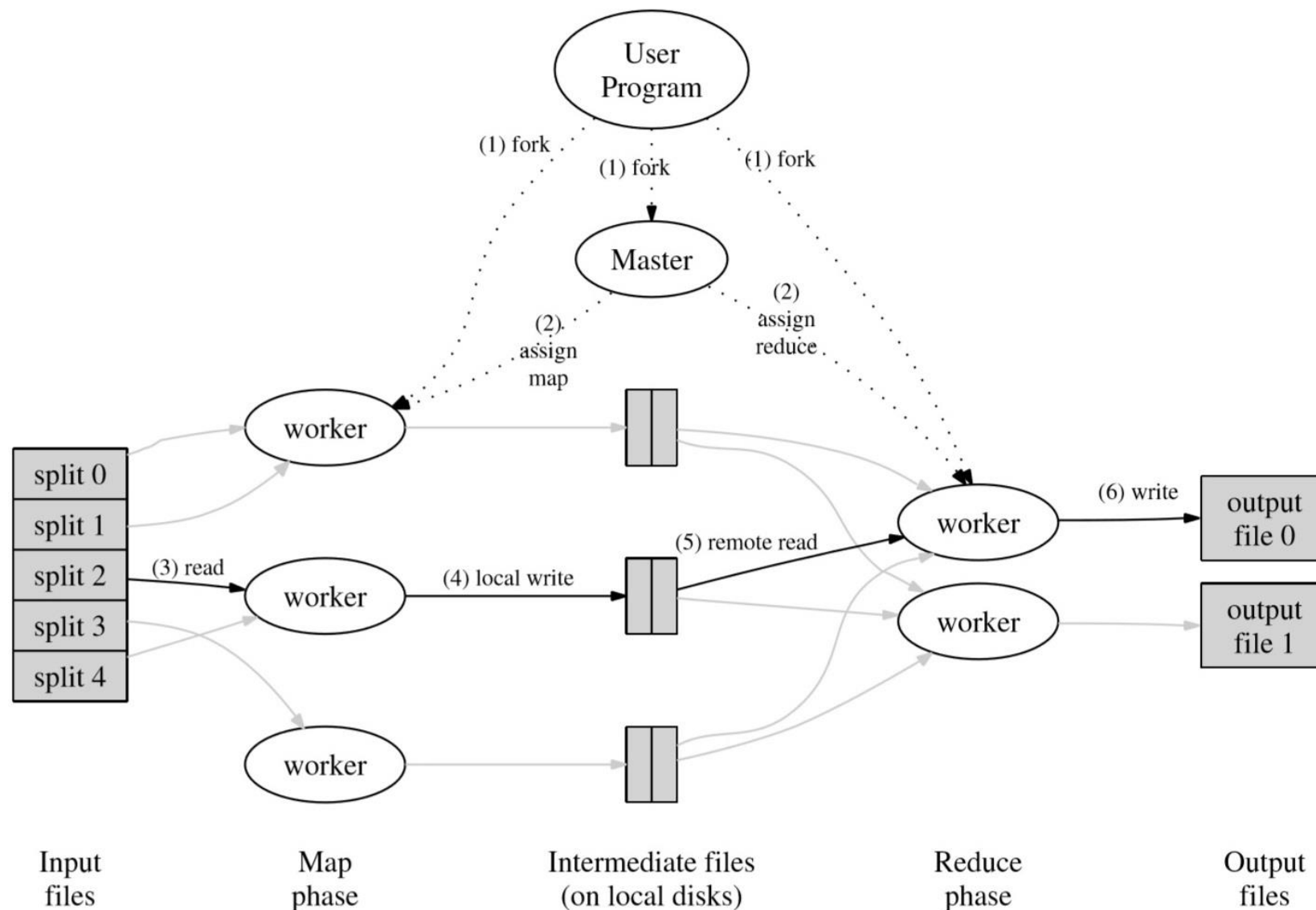
- **Grep distribuido**: el map emite una línea si se cumple con cierto patrón y el reduce solo copia las líneas intermetidas en el output.
- **Contar la frecuencia de accesos a un URL.**
- **Reverse Web-Link Graph**: la función map genera pares (*target*, *source*) por cada enlace a un URL *target* encontrado en una página *source*. La función reduce concatena la lista de URLs *source* para generar el par (*target*, *list(source)*).

- **Term-Vector por Host**: un term vector resume las palabras más importantes que aparecen en un documento como una lista de pares (*word, frequency*). La función map genera un par (*hostname, term vector*) por cada documento en el input (hostname es el URL del documento). La función reduce junta todos los term vectors y **descarta los términos infrecuentes** para generar un par (*hostname, term vector*) final.
- **Inverted Index**: la función map analiza gramaticalmente cada documento y emite una secuencia de pares (*word, document ID*). La función reduce ordena los IDs de los documentos para generar un par (*word, list(document ID)*). Se puede aumentar esta computación para **rastrear** la posición de cada palabra.
- **Distributed Sort**.

# Implementación

- La implementación de la interfaz MapReduce se puede realizar de distintas maneras dependiendo del entorno.
- Los detalles de la implementación que se presentarán a continuación está dirigida al entorno informático en uso generalizado en Google.

# Descripción de la ejecución



Cuando se llama a la función MapReduce, se ejecuta la siguiente secuencia:

1. Se divide el input en  $M$  piezas de por lo general 16 o 64 MB.
2. Una de las copias del programa corresponde al master y el resto a los workers a quienes el master les asigna el trabajo. Hay  $M$  tareas map y  $R$  tareas reduce a asignar. El master escoge workers ociosos y les asigna una tarea map o reduce.
3. El map worker lee la pieza del input que le corresponde, analiza los pares key/value y se los pasa a la función map. Los key/value intermedios se almacenan en un buffer de memoria.
4. Periódicamente, los pares en el buffer son escritos en el disco local, particionado en  $R$  regiones por una función de particionamiento. Las localizaciones de los pares almacenados en el disco son entregados al master para que se los comunique a los reduce workers.

5. Cuando se le notifica al reduce worker de las localizaciones, este usa llamadas remotas para **leer los pares del disco local** de los map workers. Una vez leída toda la información intermedia, ordena todas las keys intermedias de tal manera que se agrupen aquellos pares con las mismas keys.
6. El reduce worker itera sobre la información intermedia ordenada, y por cada key intermedia única que se encuentre, le brinda la key y el conjunto de values correspondiente a la función reduce. El output de la función **reduce es adjuntado a un archivo output** final para esta partición reduce.
7. Cuando todas las tareas se han finalizado, el master **retorna el resultado** de la función MapReduce al programa principal.

# Tolerancia al fallo

## Falla de un worker

- El maestro envía un ping periódicamente a cada worker, **si no se recibe respuesta alguna por parte de un worker** en cierto periodo de tiempo el master marca el worker como fallido.
- Las tareas en progreso del **worker fallido son reiniciadas** a su estado de espera y posteriormente asignadas a otro worker.
- Las tareas map completadas son re ejecutadas por otro worker debido a que el disco local del **worker fallido es ahora inaccesible**.
- Las tareas **reduce completadas no necesitan re-ejecutarse** pues el output se almacena en un sistema de archivos global.
- Todos **los reduce workers son notificados en caso de la re ejecución** de una tarea map para cambiar el worker de quien leen la información intermedia.

## **Falla del master**

- El master periódicamente escribe un punto de control de sus estructuras de datos que almacenan la información necesaria para dirigir los workers.
- Si la tarea del master muere, una nueva copia se puede cargar usando los puntos de control.
- Sin embargo, dado que solo hay un master y es muy poco probable que falle, se cancela el proceso MapReduce si se da una falla del master.
- Los clientes pueden revisar esta condición y reintentar la operación MapReduce si así gustan.

# Localidad

- El hecho de que la información de input se almacena en los discos locales permite ahorrar el ancho de banda de la red.
- El GFS (Google File System) divide cada archivo en bloques de 64 MB y almacena típicamente 3 copias de cada bloque en diferentes máquinas.
- El master toma la información de localidad y programa las tareas map en máquinas que contengan una réplica del input correspondiente.
- Cuando se ejecuta largas operaciones MapReduce en una fracción significativa de un clúster, la mayor parte de la información de input se lee de manera local y no se consume el ancho de banda de la red.

# Granularidad de tareas

- Como se describió anteriormente, la fase map se divide en  $M$  piezas y la fase reduce se divide en  $R$  piezas.
- Es preferible que  $R$  y  $M$  sean mucho más grandes que la cantidad de máquinas worker pues si cada worker realiza distintas tareas se mejora el balanceo de carga dinámico y además se acelera la recuperación en caso de fallos.
- En la práctica se tiende a escoger  $M$  de tal manera que cada tarea individual requiera aproximadamente 16 a 64 MB de input y  $R$  como un múltiplo pequeño de la cantidad de workers que se planea usar. Ej:  $M = 200\ 000$  y  $R = 5\ 000$  usando 2 000 workers.

# Tareas de respaldo

- Es común encontrar máquinas “rezagadas” que toman un inusualmente mayor tiempo para completar las últimas tareas map o reduce. La razón de la existencia de estas máquinas puede ser varias, por ejemplo: el tener un mal disco y experimentar un mal rendimiento en la lectura de datos.
- Para aliviar este problema, el master programa ejecuciones de respaldo de las tareas en progreso restantes cuando la operación MapReduce está cerca de terminarse. La tarea es marcada como completa cuando se culmina la tarea principal o la de respaldo.
- Experimentalmente se ha probado que esto reduce el tiempo de ejecución de largas operaciones MapReduce.

# Función de particionamiento

- Los datos se particionan en las  $R$  tareas indicadas por el usuario usando una función de particionamiento sobre las keys intermedias.
- La función de particionamiento por defecto utiliza un hash (ej:  $\text{hash}(\text{key}) \bmod R$ ).
- En algunos casos resulta conveniente particionar la información en base a otra función de las keys. Ej: Si las keys son URL y queremos que todas las entradas para un mismo host terminen en el mismo archivo output podemos usar  $\text{hash}(\text{Hostname}(\text{urlkey})) \bmod R$ .

# Función Combiner

- Útil cuando se producen repeticiones significativas de keys intermedias. Ej: en el ejemplo de contar las palabras se producirían varios pares intermedios para una misma palabra.
- En lugar de enviar todos los pares por separado a través de la red hacia una misma tarea reduce, se especifica una función combiner para unir parcialmente estos datos antes de enviarlos a la red.
- La función combiner se ejecuta en las máquinas que ejecutan una tarea map y por lo general el código es el mismo que en la función reduce.
- A diferencia de la función reduce, el output de la función combiner se almacena en un archivo intermedio a enviar a una tarea reduce.

# Omitiendo malos records

- Algunas veces se presentan algunos bugs que evitan que se puedan completar las funciones map o reduce y no pueden ser arreglados.
- A veces es aceptable ignorar unos cuantos records del conjunto de datos total.
- Cada worker se instala un manejador de señales que detecta violaciones de segmento y errores de bus. Cuando se detecta una señal, esta se envía al master. En caso el master haya recibido múltiples señales correspondientes a un record en particular, se indica que dicho record sea omitido.

# Contadores

- La librería MapReduce provee un contador para contar la cantidad de ocurrencias de un evento.
- Para usarlo se debe crear un objeto contador y utilizarlo apropiadamente en las funciones map o reduce.
- Para el ejemplo de contar la cantidad de palabras con mayúsculas tenemos:

```
Counter* uppercase;  
uppercase = GetCounter("uppercase");  
  
map(String name, String contents):  
    for each word w in contents:  
        if (IsCapitalized(w)):  
            uppercase->Increment();  
            EmitIntermediate(w, "1");
```

- Los valores de los contadores para cada worker individual son comunicados al master al momento de responder al ping de control que envía el master.
- El master suma los contadores de operaciones map o reduce exitosas y los entrega como resultado al final de la operación MapReduce.
- Algunos contadores son manejados automáticamente por la librería, como la cantidad de pares procesados en el input y pares generados como output.

# Conclusiones

- El modelo MapReduce fue usado satisfactoriamente en Google para múltiples propósitos.
- El modelo es fácil de usar pues oculta los detalles de la paralelización, tolerancia a fallas, optimización de localidad y balanceo de cargas.
- Se puede expresar una gran variedad de problemas como cálculos MapReduce.
- El modelo MapReduce es fácilmente escalable para grandes clústeres.

# Detalles a rescatar

- Restringir el modelo de programación facilita el paralelismo, la computación distribuida y el hacer dichos cálculos tolerantes a fallos.
- El ancho de banda es un recurso escaso, por ello se presenta la optimización de localidad como una solución para ahorrar el uso del ancho de banda.
- La ejecución redundante puede ser utilizada para reducir el impacto de máquinas lentas y manejar fallas de máquina y pérdida de información.